

Kapitel 3 · Eingaben prüfen

Fahrradverleih · Voraussetzung: Kapitel 1–2

Das lernen Sie

Sie formulieren Bedingungen und setzen ausschließlich if-Abfragen und while-Schleifen ein. Damit können Sie eine zu lange Rahmennummer erkennen und erneut abfragen.

1. Operatoren für Bedingungen

Allgemeine Grundlagen: Eine Bedingung ist ein Ausdruck, dessen Ergebnis über den weiteren Programmablauf entscheidet. Vergleiche wie `>` oder `==` liefern `True` oder `False`.

Merksatz – Vergleiche: `==` prüft Gleichheit; `=` speichert einen Wert unter einem Namen.

```
anzahl = 3
print(anzahl == 3) # Ausgabe: True
print(anzahl > 3)  # Ausgabe: False
```

Allgemeine Grundlagen: `and`, `or` und `not` verknüpfen oder verändern Bedingungen. Bei booleschen Bedingungen verlangt `and` zwei wahre Seiten; `or` verlangt mindestens eine wahre Seite; `not` kehrt den Wahrheitswert um. Klammern machen zusammengesetzte Bedingungen leichter lesbar.

Merksatz – Bereiche: Innerhalb eines Bereichs wird häufig mit `and` geprüft, außerhalb mit `or`.

```
monat = 13
print((monat >= 1) and (monat <= 12)) # False / Innerhalb des Bereichs
print((monat < 1) or (monat > 12))   # True
print(not (monat == 13))             # False
```

Anwendung: Fahrradverleih

Ein Vergleich liefert einen `bool`: `True` (wahr) oder `False` (falsch). Die folgenden Operatoren helfen bei der Eingabeprüfung: `==` bedeutet gleich; `!=` bedeutet ungleich.

Die Operatoren `>` und `<` bedeuten größer und kleiner (`>=` und `<=` entsprechend).

```
len(rahmenNr) > 6
```

Dieser Vergleich ist bei sieben Zeichen `True`, bei sechs Zeichen `False`.

```
len(rahmenNr) == 0 or len(rahmenNr) > 6
monat >= 1 and monat <= 12
```

Die erste Bedingung erkennt eine leere oder zu lange Rahmennummer. Die zweite beschreibt einen Monat im Bereich 1 bis 12. Ein gültiger Monat allein garantiert aber noch keinen gültigen Kalendertag.

2. if: Einmal entscheiden

Allgemeine Grundlagen: if wählt aus, ob ein Anweisungsblock ausgeführt wird. Ein Block umfasst die zusammengehörenden, gleich weit eingerückten Anweisungen. Die Einrückung ist in Python Teil der Syntax. Ist die Bedingung falsch, wird der Block übersprungen; die Ausführung geht dahinter weiter.

Merksatz – if: Eine Bedingung wird an dieser Stelle einmal geprüft. Ein danach eingerückter Block wird nur bei True ausgeführt.

```
anzahl = 0
if anzahl == 0:
    print("Keine Einträge vorhanden.")
print("Prüfung beendet.")
```

Beispielausgabe: Bei anzahl = 0 erscheinen beide Meldungen. Bei anzahl = 2 erscheint nur „Prüfung beendet.“. Die letzte Zeile gehört wegen ihrer Einrückung nicht zum if-Block.

Anwendung: Fahrradverleih

```
if len(rahmenNr) > 6:
    print("Rahmennummer zu lang!")
```

Nach der Bedingung steht ein Doppelpunkt. Der eingerückte Block wird nur ausgeführt, wenn die Bedingung True ist.

Kapitel 3 · Wiederholen mit while

3. So funktioniert die Schleife

Allgemeine Grundlagen: while ist eine kopfgesteuerte Schleife: Zuerst wird die Bedingung geprüft, dann gegebenenfalls der Block ausgeführt und danach wieder die Bedingung geprüft. Daher sind auch null Durchläufe möglich. Damit eine Eingabeschleife enden kann, muss die neue Eingabe die Bedingung verändern können.

Merksatz – while: Solange die Bedingung wahr ist, wird wiederholt. Die Bedingung beschreibt hier den Zustand, der noch korrigiert werden muss.

```
name = input("Name: ")
while len(name) == 0: #solange die Variable name leer ist
    name = input("Bitte einen Namen eingeben: ")
print("Erfasst:", name)
```

Allgemeines Beispiel: Enter ohne Text führt erneut zur Eingabe. Nach „Mia“ endet die Schleife. Wird bereits beim ersten Versuch „Mia“ eingegeben, wird der Schleifenblock kein einziges Mal ausgeführt.

Anwendung: Fahrradverleih

```
rahmenNr = input("Rahmennummer (max. 6 Zeichen): ")
while len(rahmenNr) > 6:
    print("Zu lang. Bitte erneut eingeben.")
    rahmenNr = input("Rahmennummer (max. 6 Zeichen): ")
print("Übernommen:", rahmenNr)
```

while prüft die Bedingung vor jedem Durchlauf. Solange sie True ist, wird der eingerückte Block wiederholt. Bei False geht es hinter dem Block weiter. Ist schon die erste Eingabe gültig, wird der Block überhaupt nicht ausgeführt.

Die neue Eingabe im Block ist entscheidend: Ohne sie bleibt eine zu lange Rahmennummer unverändert und die Schleife endet nicht. Ein versehentlich endlos laufendes Programm können Sie mit der Stopptaste Ihrer IDE abbrechen.

4. if und while kombinieren

Allgemeine Grundlagen: Kontrollstrukturen können ineinander stehen. while regelt die Wiederholung, eine darin stehende if-Abfrage wählt eine passende Reaktion aus. Zwei unabhängige if-Abfragen prüfen ihre Bedingungen nacheinander.

Merksatz – Kombination: Die Schleife wiederholt die Prüfung; if erklärt den konkreten Fehler. Die erneute Eingabe gehört in den Schleifenblock, damit der Zustand aktualisiert wird.

```
kuerzel = input("Kürzel (1-3 Zeichen): ")
while len(kuerzel) == 0 or len(kuerzel) > 3:
    if len(kuerzel) == 0:
        print("Das Kürzel fehlt.")
    if len(kuerzel) > 3:
        print("Das Kürzel ist zu lang.")
    kuerzel = input("Kürzel (1-3 Zeichen): ")
print("Erfasst:", kuerzel)
```

Allgemeines Beispiel: Enter, dann ABCD, dann AB erzeugt zwei Fehlermeldungen und schließlich „Erfasst: AB“. Die beiden Fehlerursachen schließen sich in diesem Beispiel gegenseitig aus.

Anwendung: Fahrradverleih

```
while len(rahmenNr) == 0 or len(rahmenNr) > 6:
    if len(rahmenNr) == 0:
        print("Die Rahmennummer fehlt.")
    if len(rahmenNr) > 6:
        print("Die Rahmennummer ist zu lang.")
    rahmenNr = input("Rahmennummer (1-6 Zeichen): ")
```

Die if-Abfragen stehen innerhalb der Schleife. Ihre Ausgaben sind deshalb eingerückt.

Kapitel 3 · Pflichtübung

Auftrag: Rahmennummer zuverlässig abfragen

Erstellen Sie `uebung_03.py`. Verwenden Sie `input()`, `print()`, `len()`, Vergleichsoperatoren, or sowie ausschließlich `if` und `while` als Kontrollstrukturen.

1. Fragen Sie eine Rahmennummer ab. Erlaubt sind ein bis sechs Zeichen.
2. Wiederholen Sie die Eingabe, solange sie leer oder länger als sechs Zeichen ist. Formulieren Sie dafür eine gemeinsame `while`-Bedingung mit `or`.
3. Geben Sie innerhalb der Schleife mit zwei `if`-Abfragen eine passende Meldung aus: „Die Rahmennummer fehlt.“ oder „Die Rahmennummer ist zu lang.“
4. Fragen Sie danach innerhalb der Schleife erneut ab.
5. Geben Sie erst nach der Schleife „Übernommen:“ und die gültige Rahmennummer aus. Ein Datenbankzugriff ist noch nicht erforderlich.

Testfolge A: Zwei Fehler hintereinander

Drücken Sie beim ersten Versuch direkt `Enter`: Es muss die Meldung über die fehlende Rahmennummer erscheinen. Geben Sie danach `ABC1234` ein: Es muss die Meldung über die zu lange Rahmennummer erscheinen. Geben Sie dann `AB1234` ein: Das Programm muss enden mit:

Übernommen: AB1234

Testfolge B: Grenzen prüfen

Starten Sie das Programm jeweils neu. `A` und `ABC123` müssen ohne Fehlermeldung akzeptiert werden. `ABC1234` muss zu einer erneuten Eingabe führen. So prüfen Sie besonders die Grenze zwischen sechs und sieben Zeichen.

Erledigt, wenn ...

Die Testfolgen funktionieren, die Einrückungen stimmen und Sie können erklären, warum die erneute Eingabe in die Schleife gehört. Eine ungültige Eingabe darf nie bei „Übernommen:“ erscheinen.

Zum Nachlesen

`if` und `while`: https://docs.python.org/3/reference/compound_stmts.html

Vergleiche und boolesche Operatoren: <https://docs.python.org/3/library/stdtypes.html#truth-value-testing>